Code In, Cluster Out

NixOS + K3s for reproducible edge Kubernetes

ces0712 • KCD Texas

Thesis

At the edge, Kubernetes reproducibility fails **long before containers start**.

If the node is part of the system, the node should be declared like the system.

> The deployment artifact is not just the image.
> It is the host, the cluster, and the workload together.

Why Edge Hurts

- constrained hardware
- unreliable networks
- awkward remote access
- imperative fixes that harden into drift

You do not want **tribal knowledge** as your recovery plan.

What Usually Fails

- playbooks automate drift, but keep a mutable baseline
- golden images help once, then age immediately
- manual kubeadm steps are hard to replay under pressure
- cluster GitOps starts too late in the stack

Containers can be reproducible while the **node is still a snowflake**.

The Model

From one operator workflow, I wanted:

- host configuration
- Kubernetes distribution
- workloads
- deployment workflow
- backup and recovery

Not five tools with five different sources of truth.

The Stack

- **NixOS** for the host contract
- **K3s** for the lightweight control plane
- **Colmena** for convergence
- **OpenTofu + Terramate** for OCI infrastructure
- **Forgejo on Raspberry Pi + Mac mini runner** for GitOps
- **RustDesk** as the real workload

Same declarative model: first on Raspberry Pis, here on real Oracle edge node.

KCD GROWING CLOUD NATIVE TOGETHER TEXAS AUSTIN

Why NixOS

- one declarative system definition
- atomic activation
- rollback by generation
- boot, users, firewall, services, and packages in one model

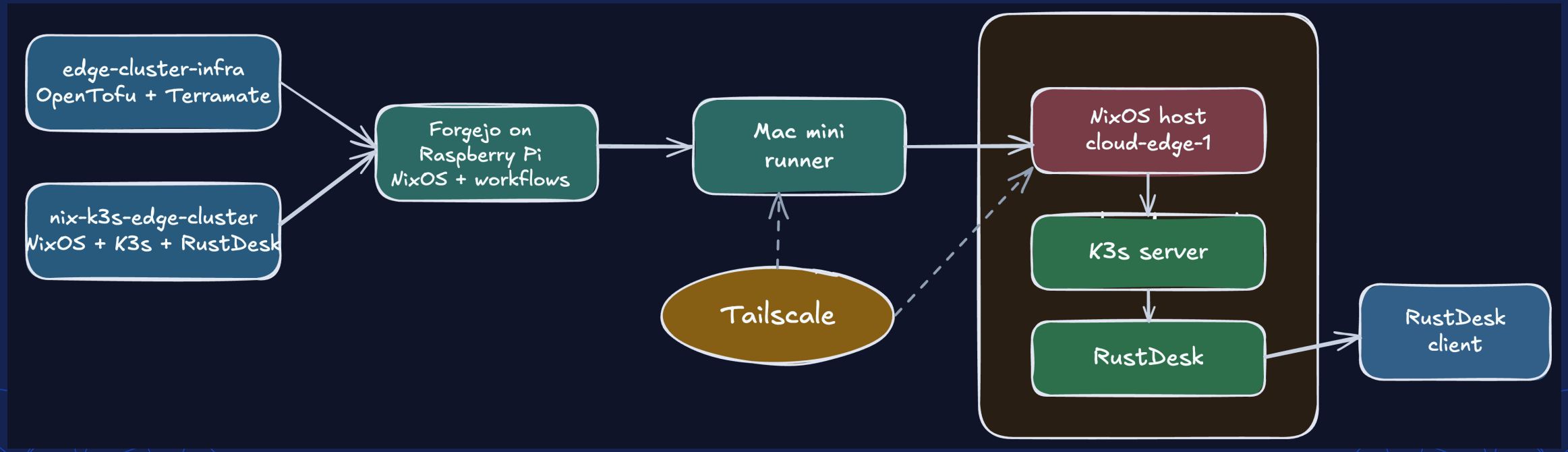
At the edge, that operational shape matters more than “cool config syntax.”

Why K3s

- small enough for edge constraints
- simple enough to recover without drama
- native NixOS module support
- workloads can be declared as part of system bootstrap

That makes the cluster feel like an extension of the host, not a second universe.

Architecture



Forgejo runs on Raspberry Pi with NixOS. Mac mini runner drives Oracle edge deploy path.

Runtime Shape

```
~/devsecops/nix-k3s-edge-cluster
  apps
    rustdesk
    default.nix
  hosts
    cloud-edge-1
      default.nix
      deployment.nix
  modules
  scripts
  LICENSE
  README.md
  flake.lock
  flake.nix
  justfile
```

This repo is the runtime contract for the node.

github.com/ces0712/nix-k3s-edge-cluster/blob/main/hosts/cloud-edge-1/default.nix

```
edgeCluster = {
    bootstrap = {
        enable = false;
        permitRootLogin = false;
    };

  apps.rustdesk = {
      enable = true;
      serverHost = "cloud-edge-1";
      dataDir = "/srv/edge-cluster/rustdesk";
  };

  backup = {
      enable = true;
      stateDir = "/srv/restic-backup";
      paths = [
          "/srv/edge-cluster/rustdesk"
          "/var/lib/rancher/k3s/server/token"
      ];
  };

};
```

Real Workload

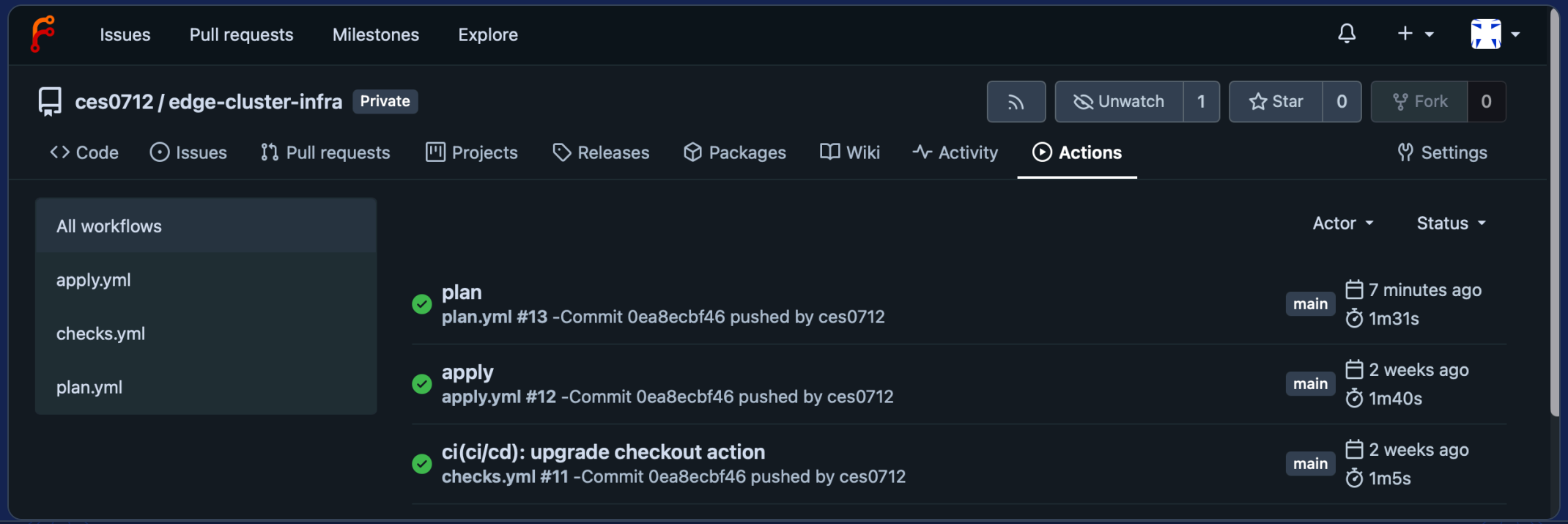
```
services.k3s.manifests.rustdesk = {
  target = "rustdesk.yaml";
  content = [{
    kind = "Deployment";
    spec.template.spec = {
      hostNetwork = true;
      dnsPolicy = "ClusterFirstWithHostNet";
      volumes = [{
        name = "rustdesk-data";
        hostPath.path = cfg.dataDir;
      }];
      containers = [
        { name = "hbbs"; args = ["hbbs" "-r" "...:21117"]; }
        { name = "hbbr"; args = ["hbbr"]; }
      ];
    };
  }];
};
```

What it proves

- workload declared inside Nix
- K3s bootstraps with deployment intent already present
- `hostNetwork = true` matches the edge access model
- `hostPath.path = cfg.dataDir` keeps state on the node
- two containers define the real RustDesk server path

github.com/es0712/nix-k3s-edge-cluster/blob/main/apps/rustdesk/default.nix

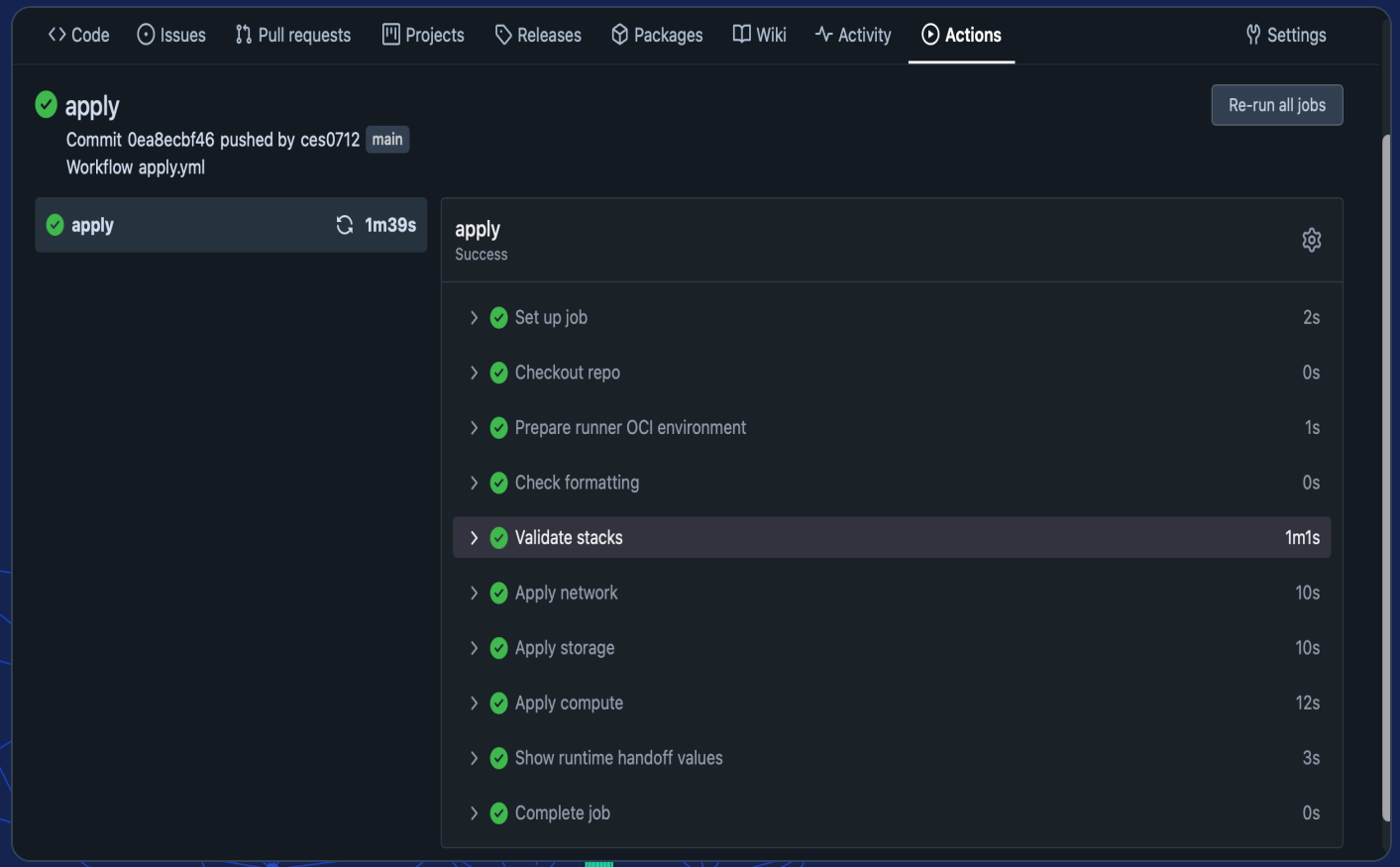
GitOps Path



What it proves

- repo has checks, plan, apply workflows
- infra path visible in Forgejo
- Forgejo hosted on Raspberry Pi, runner on Mac mini

GitOps Apply



What it proves

- validate stacks before apply
- network, storage, compute converge in order
- handoff values come out of infra workflow
- not one fake green job, real stages

The Hard Parts

- bootstrap, reboot, and bootloader correctness
- OCI public IP and Tailscale tradeoffs
- secrets with `sops-nix`
- K3s module limits at the edge
- centralized local OpenTofu state

This is where real learning was.

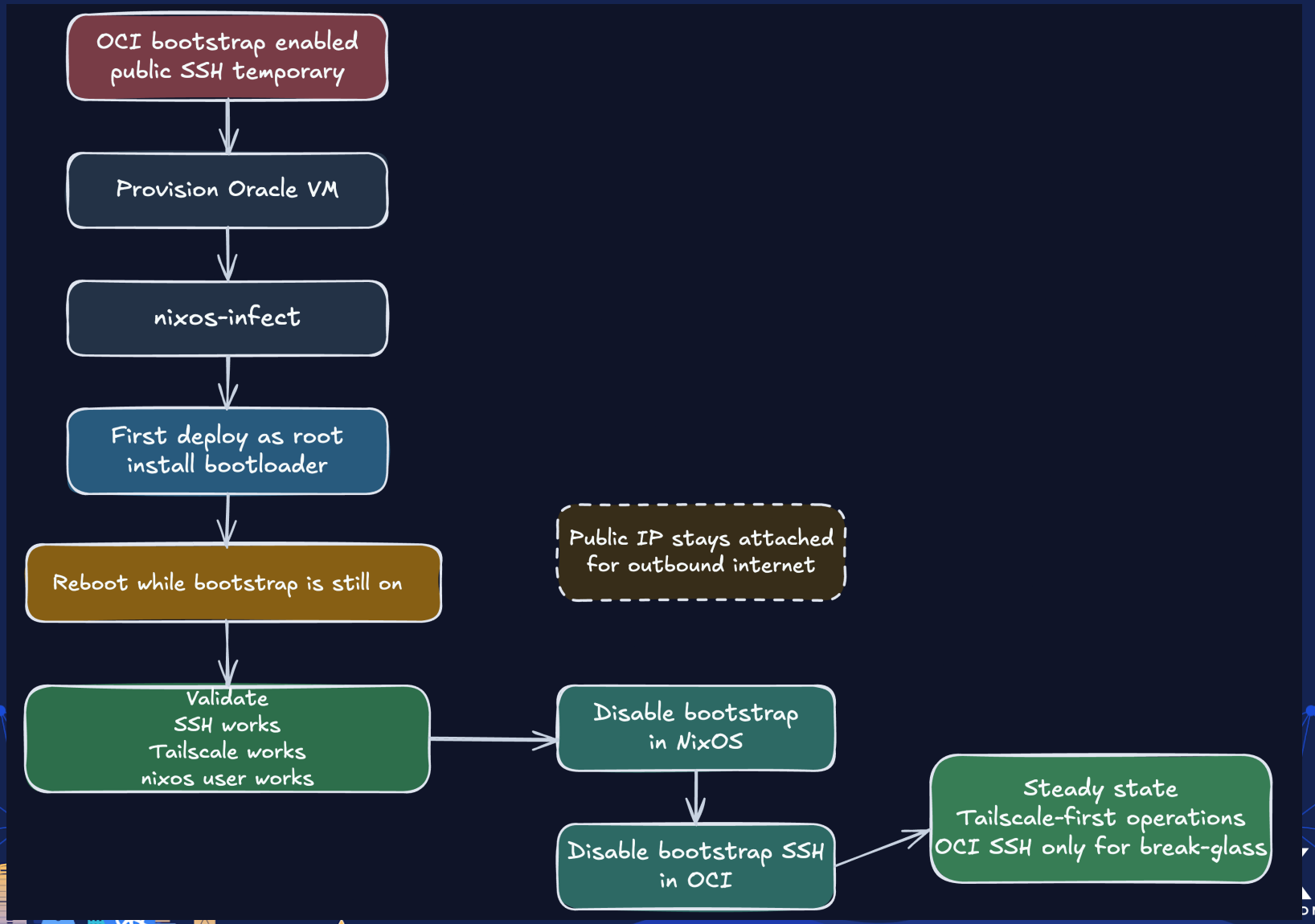
Bootstrap reality

The hard part was not Nix syntax.

It was **operational sequencing**.

- temporary OCI bootstrap SSH
- first root deploy installs bootloader
- only then disable bootstrap access

Bootstrap Flow



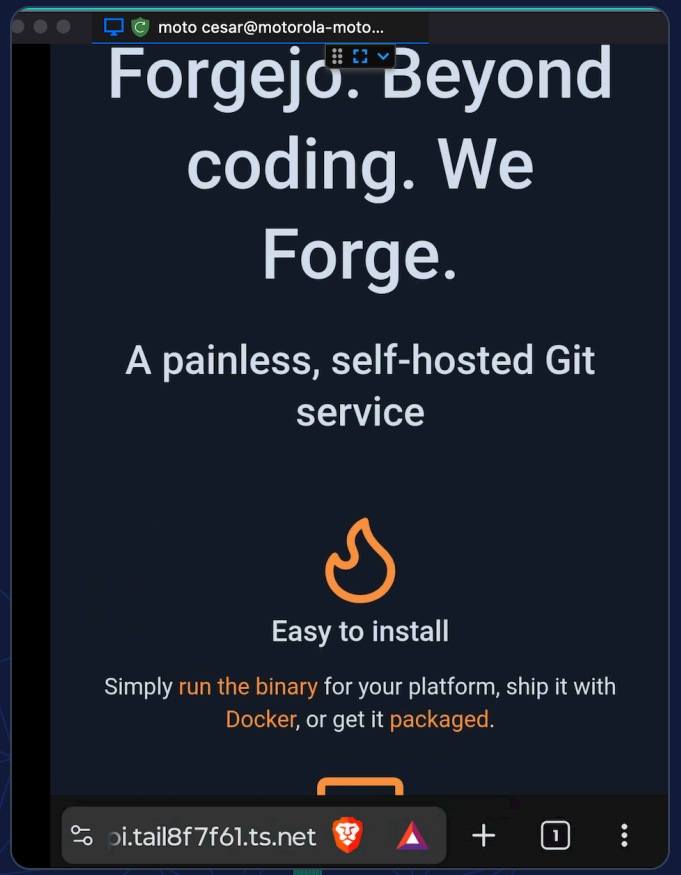
Cluster Proof

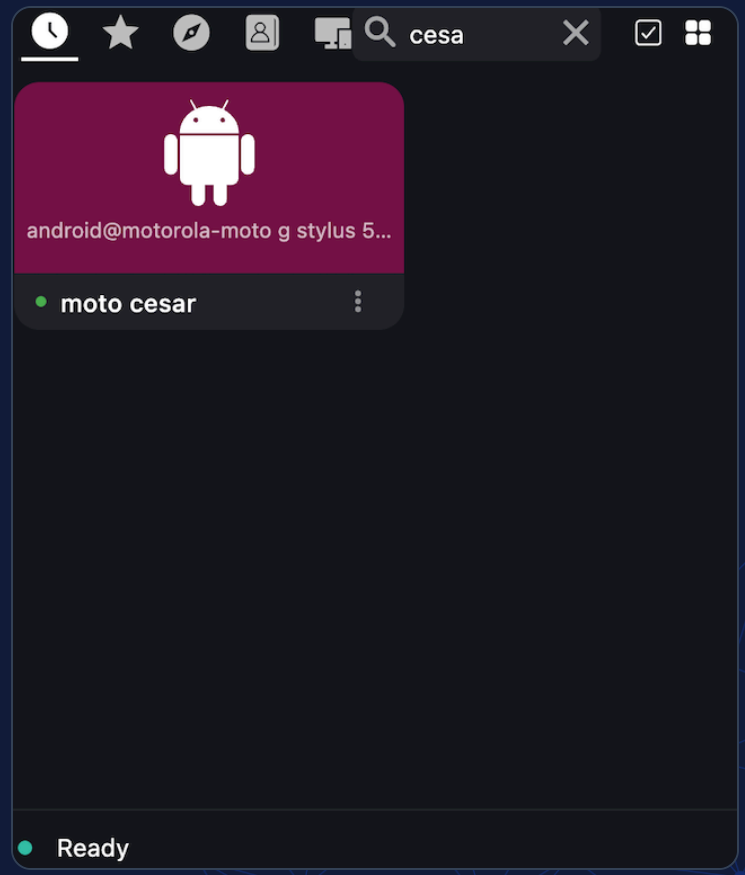
```
Context: default [RW]                                  <a>       Attach          <f> Show F
Cluster: default                                       <?>       Help
User:    default                                       <l>       Logs
K9s Rev: v0.50.18                                      <p>       Logs Previous
K8s Rev: v1.34.5+k3s1                                  <shift-f> PortForward
CPU:     3%                                            <s>       Shell
MEM:     21%
──────────────────────────────────────── containers(rustdesk/rustdesk-server-846846db57-k7
 IDX↑  NAME  PF IMAGE                          READY STATE    RESTARTS PROBES(L:R:S)
 M1    hbbs  ● rustdesk/rustdesk-server:latest true  Running         1 off:off:off
 M2    hbbr  ● rustdesk/rustdesk-server:latest true  Running         1 off:off:off
```

What it proves

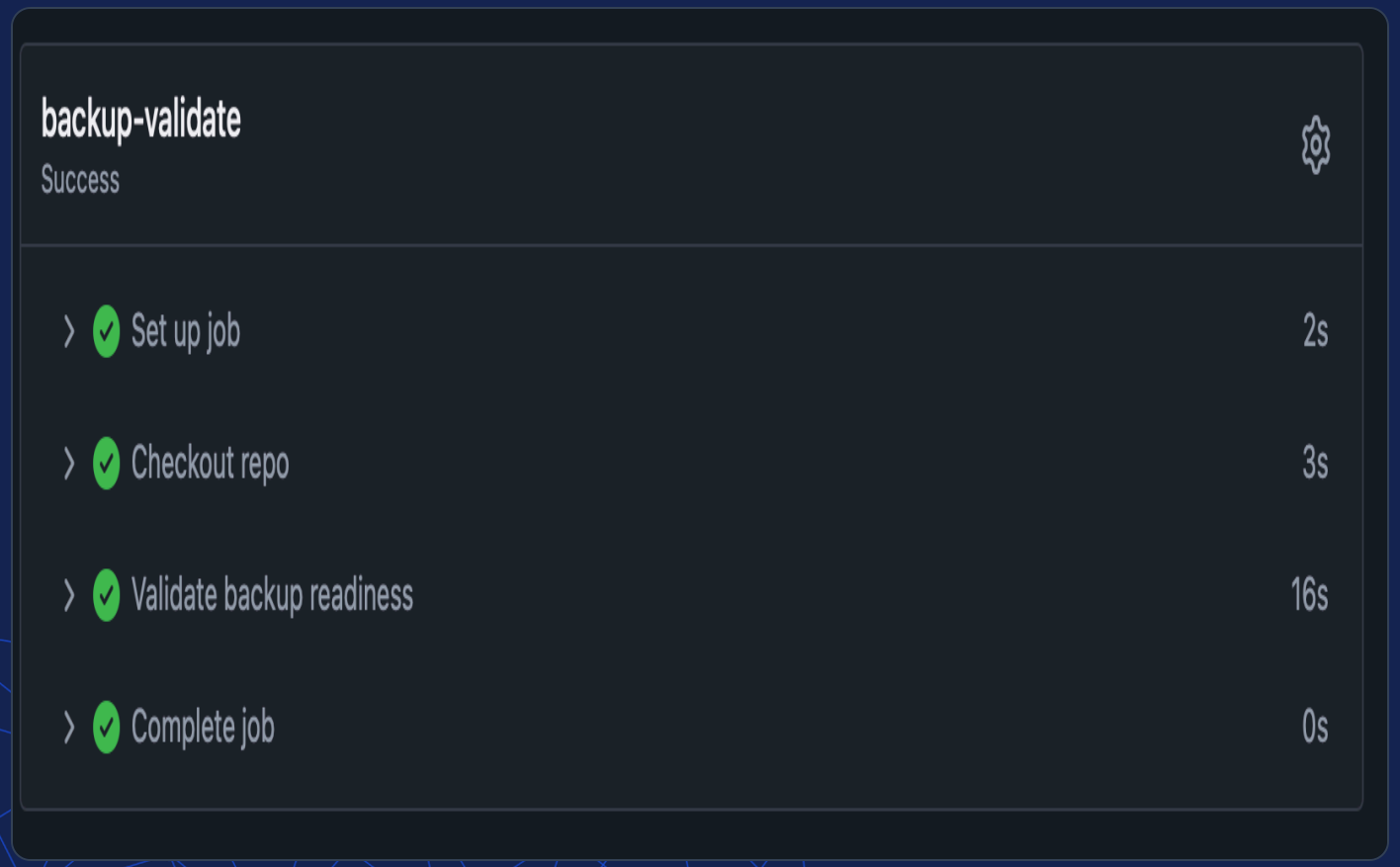
- `hbbs` and `hbbr` both running
- Nix-defined workload reached cluster intact
- runtime matches code shown earlier

User Proof


moto cesar@motorola-moto...
Forgejo. Beyond coding. We Forge.
A painless, self-hosted Git service
Easy to install
Simply run the binary for your platform, ship it with Docker, or get it packaged.
i.tail8f7f61.ts.net


cesa
android@motorola-moto g stylus 5...
moto cesar
Ready

Recovery



What it proves

- backup readiness is visible in automation
- repository access is validated before emergency day
- restore confidence is operational, not tribal knowledge

> Reproducibility is not real until **recovery is boring**.

Use It When

Strong fit

- remote edge nodes
- reproducible rebuilds
- multi-arch or weird hardware
- small fleets with high discipline

Maybe not

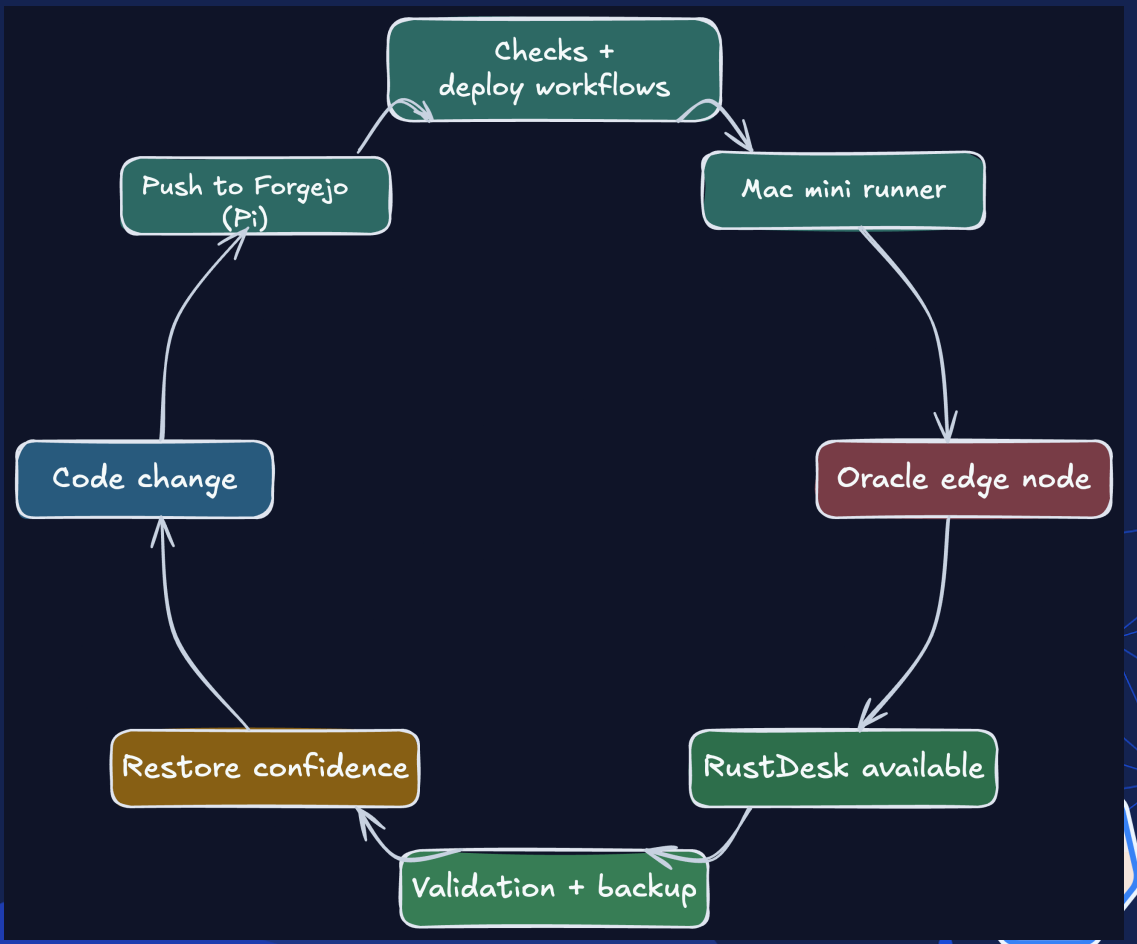
- you only need app GitOps
- your team needs the lowest-friction tools
- the node itself is disposable
- Nix complexity is not justified

Final Thought

The most interesting thing here is not Nix.

It is moving source of truth **down to node boundary**.

If you can declare host, cluster, and workload together, you get different kind of reliability.



Thank You

- NixOS + K3s
- OCI + Tailscale
- Forgejo on Raspberry Pi + Mac mini runner
- RustDesk as proof point

Repos

- github.com/ces0712/infrastructure-nixos
- github.com/ces0712/nix-k3s-edge-cluster

Article

- dev.to/ces0712/code-in-cluster-out

Kubernetes at the Edge: Declaring the Indeterminable with Nix and K3s